AI CAPSTONE PROJECT | FINAL REPORT

ESBC-AI

Easy Service for Bangladeshi Citizens: a bilingual, citation-grounded AI assistant

Farjana Ferdausi | Ostad AI & ML Engineering Bootcamp, Batch 6, Module 27 | 28 October 2026

github.com/farjanaferdausi-cs50ai/ESBC-AI | esbc-ai.vercel.app | esbc-ai-api.onrender.com/docs

1. Problem solved

Information about Bangladeshi government services is spread over many official portals. Pages are often written in one language only, some are rendered with JavaScript, and fees and rules change without much notice. A citizen who wants to register a death, correct an NID, renew a passport or apply for an allowance has to read several pages, and a wrong fee or a missing document can cost days or money.

I built ESBC-AI to give citizens one place to ask these questions in Bangla or English. Its three goals are: (1) every answer must come from an official **.gov.bd** page, (2) every answer must show the page it came from and the date I checked it, and (3) when the sources do not cover a question, the system must say it is not sure instead of guessing. I treat the third goal as the most important, because a confident wrong answer about a legal fee is worse than no answer.

2. Solution overview

ESBC-AI is a retrieval-augmented generation (RAG) assistant with a React front end, a FastAPI back end and a LangGraph agent. The knowledge base contains **29 Markdown documents (76 chunks)** that I collected from official pages. Each document stores its source URL, the date I checked it, its service area and its language. Many Bangla pages have a companion English translation, clearly labelled as my own translation, with the same URL and date.

Services covered: death registration, birth registration, birth and death fees (Registrar General, updated 4 April 2026), e-passport (fees updated 12 July 2026), National ID (an older 2015 fee notice, flagged as old in the answer), tax (contact numbers only) and Department of Social Services allowances (old age, widow, disadvantaged communities, serious disease support). A BDRIS maintenance notice (19 to 25 October 2026) is also stored so the assistant can warn users about it.

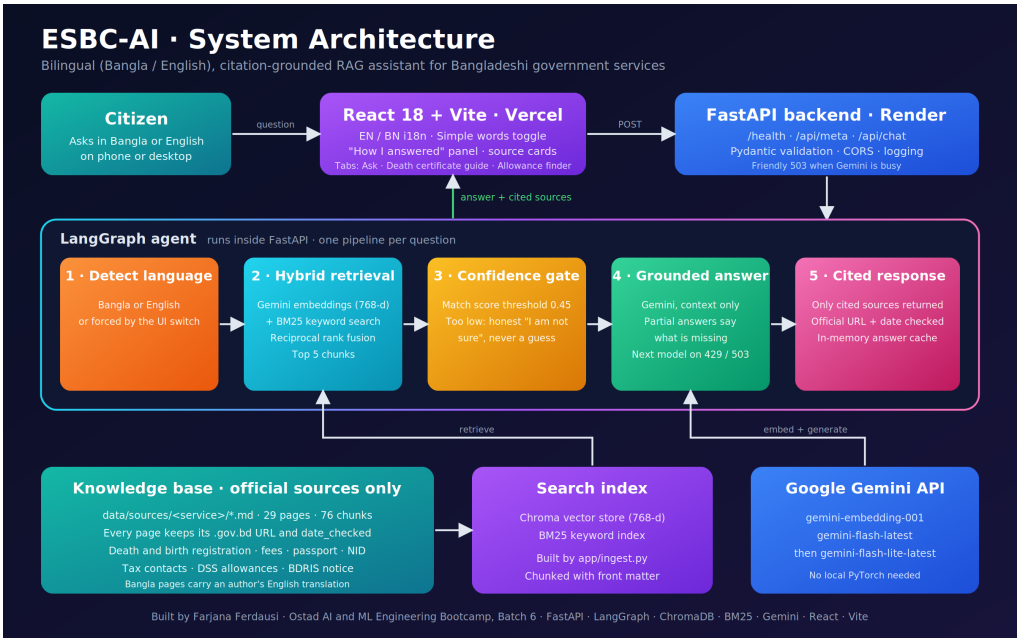


Figure 1. System architecture. The same diagram is in the README.

3. AI workflow and design decisions

Each question goes through a LangGraph pipeline with five steps:

- **Language detection.** The language is detected from the question, or forced by the language switch in the interface (Auto, English, Bangla).
- **Hybrid retrieval.** I combine dense search (Gemini embeddings, 768 dimensions, stored in ChromaDB) with BM25 keyword search and merge both rankings with reciprocal rank fusion (top 5). Dense search handles paraphrases and cross-language matches; BM25 catches exact terms such as fee names and portal names that embeddings can blur.
- **Confidence gate.** If the best match score is below a threshold (0.45), the system refuses without calling the language model.
- **Grounded generation.** Gemini answers only from the retrieved context, gives a partial answer and says what is missing, dates older figures, and mentions closures. If the context says nothing about the asked service, it must reply NO_ANSWER. If one model returns a 429 or 503 error, the agent tries the next model in a configurable list.
- **Cited response.** Only the sources that the answer actually cites are returned, with URL and date checked. The interface shows them as source cards and also shows a 'How I answered' panel (language, searched pages with match scores, gate result).

Key design decisions:

- **Official, dated sources only.** I never filled a gap from news sites, blogs or memory. I also kept a list of fees and thresholds that came from a second assistant and did not put any of those numbers into the knowledge base because I had not confirmed them on the official page. They are listed under future work.
- **Gemini embeddings instead of BGE-M3 or e5.** My development PC uses Windows Application Control, which blocks PyTorch. I therefore used the Gemini embedding API (gemini-embedding-001). This differs from my proposal and also keeps the deployed server light enough for a free hosting plan.
- **Two-layer refusal.** The first layer is the retrieval score gate; the second is the model rule to answer NO_ANSWER when the context does not cover the question. The evaluation showed why two layers are needed (Section 5).
- **Bilingual by design.** Bangla questions and English questions both work against Bangla and English sources, and the user can force the answer language. A 'Simple words' mode asks the model for very plain wording.
- **Resilience.** The server returns a friendly 503 when the model is busy, the front end waits while the free server wakes up, and an in-memory cache answers repeated questions quickly.

4. Implementation

Layer	What I built
Front end	React 18 and Vite, custom CSS, English and Bangla interface text, tabs (Ask, Death certificate guide, Allowance finder), service tiles, source cards, copy and print buttons, animated network background, deployed on Vercel.
Back end	FastAPI with /health, /api/meta and /api/chat, Pydantic validation, CORS, logging and friendly error handling, deployed on Render through a render.yaml blueprint (auto-deploy on git push).
AI	LangGraph agent, Gemini embeddings and generation, ChromaDB, rank-bm25, reciprocal rank fusion, confidence gate, citation filtering, answer cache.
Quality	22 offline pytest tests (fake embeddings), scripts/doctor.py environment check, eval/run_eval.py, scripts/update_readme.py to keep README links, results and screenshots in sync.

Layer	What I built
Repository	Public GitHub repository with README (setup, AI methodology, usage, screenshots, architecture diagram), MIT licence and regular commits.

5. Results

I wrote a question set of 29 questions (answerable and out of scope, Bangla and English) with keyword-based checks. The keywords come from the stored official sources. The latest run completed with no errors:

Metric	All	English	Bangla
Retrieval hit rate (top 5)	100%	100%	100%
Answer accuracy (keyword match)	100%	100%	100%
Refusal accuracy on out-of-scope questions	100%		

How I read these numbers. The checks are keyword based, not an LLM judge, and the question set is small and was written by me after I built the knowledge base. The result is therefore a lower bound on quality and an optimistic estimate of real-world accuracy. An earlier run, before I added English translations of Bangla pages, lowered the gate threshold (0.6 to 0.45) and changed the prompt to allow partial answers, scored 73.9% retrieval and 60.9% answer accuracy (English 46.2%, Bangla 80%). Most failures were English questions against Bangla-only pages. Adding companion translations fixed that, which shows that language coverage of the sources matters more than the model.

Retrieval scores do not fully separate in-scope from out-of-scope questions. The lowest best-score among answerable questions was 0.654, and the highest among out-of-scope questions was 0.664. A single threshold cannot separate them perfectly, so the second refusal layer (the model's NO_ANSWER rule) is necessary. For example, when I asked about the driving licence fee, which is not in the knowledge base, the system answered that it was not sure instead of inventing a fee.

Manual checks. A death registration checklist returned the documents, the three fee tiers (free within 45 days, 25 taka up to 5 years, 50 taka after 5 years) with the 4 April 2026 date, and numbered citations. The old age allowance question returned 65 years for men and 62 years for women with the DSS source.

6. Proposal versus delivered

Proposal item	Status	Note
Cited Q&A over official sources	Delivered	29 documents, URL and date on every source
Honest refusal	Delivered	Score gate plus NO_ANSWER rule
Hybrid retrieval (dense + BM25 + RRF)	Delivered	Gemini embeddings instead of BGE-M3 / e5
Bilingual answers, language switch, simple words	Delivered	Auto, English, Bangla and a plain-words toggle
Death certificate guide, allowance finder	Delivered (light)	Question-based guides, not form filling
FastAPI back end, tests, deployment	Delivered	Vercel and Render; 22 tests
Cross-encoder re-ranker	Not built	Planned as a later improvement
RAGAS evaluation	Not built	Custom keyword metrics used instead
Form filling, OCR, application packet PDF	Not built	Out of time
Case tracker and payment simulation	Not built	Out of time
Memory vault, Langfuse / LangSmith tracing	Not built	Out of time
10 synthetic documents	Not built	Out of time
14 service areas with data	Partial	7 of 14 service areas have data

7. Limitations

- **Coverage.** Only 7 of 14 planned service areas have data. Land mutation, driving licence, police clearance, trade licence and utility connections are not covered, so the assistant correctly declines those questions today.
- **Freshness.** Fees and rules change. The NID fee notice in the knowledge base dates from 2015 and is flagged as old. Every answer shows the date I checked the source, but I do not monitor the portals automatically.
- **Free-tier limits.** Gemini's free tier allows only a few generations per minute and often returns 503 errors at busy times. My evaluation runs failed several times for this reason, and the live demo can be slow. The Render free server also sleeps, so the first request can take up to a minute.
- **Evaluation.** Keyword matching can miss a correct answer in different words, and it cannot detect a wrong sentence next to a right keyword. The question set is small and self-written. A human review and a larger set (the proposal asked for 40 questions) are still needed.
- **Not an official service.** The interface says so, and asks users to confirm with the relevant office before applying or paying.

8. Roadmap

- Verify and add the remaining services from official pages (tax and e-TIN steps and thresholds, current NID fees and corrections, DSS application windows, passport correction rules, land mutation, driving licence, police clearance, utilities). A figure enters the knowledge base only after I confirm it on the official page and record the URL and date.
- Grow the evaluation set to 40 questions, add an LLM or human judge, and track retrieval and answer quality over time.
- Add a cross-encoder re-ranker, tracing, a paid or second API key for reliable generation, and pre-computed answers for common questions.
- Build the proposal's P2 and P3 features: case tracker, printable application packet, correction pathway guide and OCR.

9. Reproducing the project

The README has full setup instructions. In short, with a Gemini API key in `backend/.env`:

- `python -m app.ingest` builds the Chroma index and the BM25 corpus from `data/sources` (29 files, 76 chunks).
- `python -m pytest -q` runs the 22 offline tests with fake embeddings, so they need no API key.
- `python eval/run_eval.py` runs the 29-question evaluation and writes `docs/evaluation.md`.
- `npm run dev` in `frontend/` starts the interface; `VITE_API_URL` points it at the API.

10. Conclusion

ESBC-AI shows that a small, carefully sourced RAG system can give citizens clear, cited answers in their own language and, more importantly, can admit what it does not know. The main lessons were that source language coverage and source quality matter more than model choice, that a refusal needs more than one layer, and that free-tier infrastructure needs deliberate engineering for retries, caching and waking servers. The system is deployed, documented and open source under the MIT licence, and I have listed honestly what I did not build.